

SamurAI — Content Pack (LinkedIn / Blog / X)

Everything below is drawn from the actual build. Framing leads with the engineering and responsible-use story (not "uncensored/jailbroken"), which is what reads as credible to security engineers and hiring managers.

1. LinkedIn Post (copy-paste ready)

Just built SamurAI: a self-hosted, AI-assisted web application security scanner for authorized penetration testing and bug-bounty work. Start to working tool in about a day.

The goal was to run powerful open-weight LLMs on my own GPU instead of fighting rate limits and generic-assistant refusals on legitimate offensive-security questions I'm authorized to ask. So I provisioned an NVIDIA H200 (141GB VRAM) on NVIDIA Brev and went to work.

The journey was the real lesson. My first serving stack, vLLM, crashed hard on the instance's bleeding-edge CUDA 13.0 / Driver 580: FlashAttention 3 was incompatible with the compiled kernels, and both FlashInfer and enforce-eager died during CUDA graph capture. I pivoted to Ollama, whose packaged runtime handled the driver stack cleanly. Takeaway number one: on a bleeding-edge CUDA stack, a packaged runtime can beat a from-source server.

The architecture is dual-model. WhiteRabbitNeo 33B (a cybersecurity-specialized model) handles vulnerability analysis and the interactive security chat; Qwen2.5-72B writes clean, professional reports. Both are Q4_K_M GGUF quants, ~66GB combined, running side by side on one GPU. A specialist plus a generalist beats one model doing everything.

Lesson two, learned the hard way: model refusal behavior lives in the weights, not just the system prompt. And lesson three: framework churn is half the battle. I debugged Gradio 6 breaking changes live before the four-tab UI finally returned HTTP 200 on 7860.

Next up: integrating the nuclei engine with a local-LLM triage layer, CVSS/CWE mapping, and a mandatory scope-and-authorization gate before any scan runs. For a security tool, that gate isn't polish, it's the difference between a credible engineer and a liability.

For authorized security testing only.

#CyberSecurity #AppSec #LLM #BugBounty #GPU #AIEngineering

2. Technical Blog Post

Building SamurAI: Self-Hosting Security LLMs on an H200 for Authorized Pentesting

I spend my days in the security industry, and one recurring friction point has always bugged me: general-purpose hosted AI assistants tend to refuse to discuss exploitation techniques — even when you're the one holding the signed scope document. For authorized penetration testing and bug-bounty work, that refusal isn't a safety feature, it's a workflow blocker. So over roughly one day I built **SamurAI**, a self-hosted, AI-assisted web application security scanner for *authorized* testing. This is the engineering write-up: the infrastructure, the architecture decisions, the failures, and the lessons.

Why Self-Host the Models

The core motivation was control. I wanted to run powerful open-weight LLMs on my own GPU instead of paying per token and being rate-limited and filtered by commercial APIs. For legitimate offensive-security work, that matters twice over: once for cost and latency, and once because security-domain open-weight models will actually engage with the material a hosted assistant won't. Self-hosting also means my scan data — target details, findings, draft reports — never leaves infrastructure I control.

The important nuance, and the thing I kept coming back to: capability is only half the story. A tool that will discuss exploitation has to be paired with authorization controls, or it's a liability instead of an asset. More on that at the end.

The Infrastructure Journey: Brev, an H200, and a vLLM Faceplant

I provisioned an **NVIDIA H200** instance on NVIDIA Brev — 141GB of VRAM, which is the detail that makes the whole dual-model design possible later.

My first instinct for model serving was **vLLM**. It's fast, it's the default recommendation, and it should have been boring. It was not. The instance shipped a bleeding-edge stack — **CUDA 13.0, Driver 580** — and vLLM's compiled kernels did not survive contact with it. FlashAttention 3 was incompatible with the compiled kernels, and when I fell back to the FlashInfer backend, it crashed during CUDA graph capture. Disabling graph capture with enforce-eager mode crashed too. I was chasing a compatibility problem three layers down from my actual project.

So I pivoted to **Ollama**, and it handled the driver and runtime compatibility cleanly on the first try. The lesson wrote itself: on a bleeding-edge CUDA stack, a packaged runtime beat a from-source server. vLLM will almost certainly catch up, but on day zero of a new driver, Ollama's packaged runtime was simply more robust.

A couple of smaller battles along the way, for anyone who'll hit them too:

- Ubuntu's "externally-managed environment" blocked `pip` the way it does on every modern distro now. Fixed with `python3-venv` and a dedicated virtualenv — the correct answer, not `--break-system-packages`.
- I worked over SSH tunnels, and the instance's SSH daemon rate-limited and briefly IP-banned rapid reconnects, fail2ban-style. Annoying during iterative debugging, and a decent argument for standing up a public share URL for demos instead of hammering SSH.

Dual-Model Architecture — and Why Quantization Makes It Fit

The design decision I'm happiest with is running **two models side by side**, a specialist and a generalist, each doing what it's best at:

- **WhiteRabbitNeo 33B** — a cybersecurity-specialized open-weight model. This drives vulnerability analysis, exploit-path suggestions, and the interactive security chat. Pulled as a GGUF `Q4_K_M` quant via Ollama, about 19GB on disk.
- **Qwen2.5-72B-Instruct** (a community build) — a large generalist I use for writing clear, professional bug-bounty and pentest reports. Also GGUF `Q4_K_M`, about 47GB.

Here's where the H200's 141GB earns its keep. `Q4_K_M` quantization is the enabling trick: four-bit quantization shrinks roughly 100B parameters' worth of models down to about 66GB combined, so a specialist and a generalist coexist comfortably on one GPU with headroom to spare. Without quantization, neither of these would fit alongside the other.

One finding worth stating plainly, because it surprised people when I described it: I tested whether a stock instruct model plus a "system-prompt override" could stand in for a security-tuned model. It can't. The refusal behavior is baked into the weights, not layered on at the prompt. No amount of system-prompt coaxing changes that — which is exactly why the project uses purpose-built security weights rather than trying to coax a general model. And again: those weights are paired with authorization controls, not turned loose.

I also stood up **Open WebUI** on the box — a ChatGPT-style front end — for ad-hoc model use outside the scanner.

A Tour of the Tool

Samurai is Python and deliberately modular:

- `tools/recon.py` — HTTP security-header analysis, technology-stack fingerprinting, DNS enumeration with subdomain brute-forcing, an endpoint crawler that extracts forms, parameters, JS files, and API endpoints, and sensitive-path discovery across ~45 common paths (think `.env`, `.git`, backup files, admin panels).
- `tools/vuln_scanner.py` — checks for reflected XSS (GET and POST), SQL injection (error-based and boolean-blind), SSTI, LFI, open redirect, and CORS misconfiguration.

- `agents/pentest_ai.py` — the orchestration layer. It pipes scan results to WhiteRabbitNeo for analysis and exploit suggestions, and to Qwen-72B for report writing. It also powers the interactive chat and a payload generator covering 14 vulnerability types.
- `app.py` — a Gradio web UI with four tabs: Scanner, Bug-Bounty Reporter, AI Chat, and Payload Generator, served on port 7860.

It ships with a Dockerfile, a one-command `deploy.sh`, a README, and `requirements.txt` so the whole thing is reproducible.

Debugging War Story: Gradio 6 Broke Everything at Once

Deployment was the usual `scp` the project to the H200, build a venv, install dependencies. Then I ran straight into **Gradio 6 breaking changes**, and had to debug them live against a running box:

- `Textbox` dropped `show_copy_button`.
- `theme` and `css` moved off the `Blocks` constructor and onto `launch()`.
- `ChatInterface` no longer accepts a `type` argument.
- Chat history now arrives as a list of message dicts instead of tuples — so the handler had to be rewritten to read the new shape.

None of these are hard individually. Together, on a framework version bump, they're death by a thousand cuts — the app refuses to even construct until you've found all four. The end state was worth it: app live on the H200, both models loaded, Gradio returning HTTP 200 on 7860.

Lessons Worth Sharing

1. **On bleeding-edge CUDA, a packaged runtime (Ollama) beat a from-source server (vLLM).**
Pick your battles with the GPU stack.
2. **Q4_K_M GGUF quantization is what lets ~100B params of models share one GPU.** It's the difference between "pick one model" and "run both."
3. **Specialist + generalist beats one model doing everything.** A security-tuned model for analysis, a strong generalist for prose.
4. **Model refusal lives in the weights, not just the prompt.** Choose the right weights instead of fighting the wrong ones.
5. **Framework churn is half the battle.** Budget real time for breaking changes — they don't announce themselves until build time.
6. **For a security tool, authorization controls and responsible-use framing aren't optional polish.** They're what separates a credible engineer from a liability.

What's Next

From a six-lens review of the project, the roadmap I'm most excited about:

- Integrate the **nuclei** engine (9,000+ community templates) with a local-LLM triage layer that dedupes and ranks findings.
- **CVSS v3.1 scoring** plus CWE and OWASP mapping on every finding.
- A **severity dashboard** with KPI tiles and charts.
- **Live-streaming the model's reasoning** into the UI so you can watch it work.
- A **mandatory scope and authorization gate** — an allowlist plus an attestation step — before any scan runs. Of everything on this list, this is the clearest professional-maturity signal, and it's the next thing I'm building.

Responsible Use

Samurai is built for **authorized security testing only** — penetration tests you've been contracted for and bug-bounty programs whose scope you're operating inside. Running any scanner against systems you don't own or lack explicit written permission to test is illegal and unethical, full stop. The capability to engage with exploitation techniques exists here to serve testers who already hold authorization, and the roadmap's scope-and-attestation gate is there precisely to keep that line bright. Use it the way a professional would, or don't use it at all.

3. X / Twitter Thread

1/ I spent ~a day building Samurai: a self-hosted, AI-assisted web app security scanner for *authorized* pentesting & bug bounty. Two open-weight LLMs on one GPU. Here's the engineering story — including the parts that broke. 🛠️

2/ Why self-host? For authorized offensive-security work, general-purpose assistants refuse to engage with techniques you're contracted to use. So I ran security-specialized open-weight models on my own GPU instead of fighting API filters + rate limits.

3/ Hardware: an NVIDIA H200 (141GB VRAM) on NVIDIA Brev. My first serving stack was vLLM — and it faceplanted on the box's bleeding-edge CUDA 13.0 / Driver 580. FlashAttention 3 clashed with the compiled kernels; FlashInfer + enforce-eager both died in CUDA graph capture.

4/ Pivoted to Ollama. Clean on the first try. Lesson 1: on a day-zero CUDA stack, a packaged runtime can beat a from-source server. Sometimes the fast path is the robust one.

5/ Architecture is dual-model: a specialist + a generalist. WhiteRabbitNeo 33B drives vuln analysis + security chat; Qwen2.5-72B writes the professional reports. Both Q4_K_M GGUF, ~66GB combined.

6/ Q4_K_M quantization is the whole trick — 4-bit shrinks ~100B params of models enough that both fit on one GPU with headroom. Lesson 2: model refusal lives in the *weights*, not the prompt. Pick the right weights instead of fighting the wrong ones.

7/ The tool (Python): recon (headers, tech fingerprint, DNS/subdomains, crawler, path discovery), a vuln scanner (XSS/SQLi/SSTI/LFI/redirect/CORS), an AI orchestration layer, and a 4-tab Gradio UI. Shipping it meant debugging a wall of Gradio 6 breaking changes live.

8/ Next: nuclei engine (9k+ templates) + a local-LLM triage layer, CVSS/CWE scoring, a severity dashboard, and a mandatory scope + authorization gate before any scan runs.

For authorized security testing only — test only what you own or have written permission to assess.

#CyberSecurity #AppSec #LLM #BugBounty